

Лабораторная работа №4

Задание для самостоятельного выполнения

Шубнякова Дарья НКНбд-01-22

1. Вводная часть

2. Основная часть

3. Результаты

1. Вводная часть

1.1 Цели и задачи

Самостоятельно реализовать модель в NS-2 и наглядно продемонстрировать результаты в Xgraph и GNUPlot. 1. Для приведённой схемы разработать имитационную модель в пакете NS-2. 2. Построить график изменения размера окна TCP (в Xgraph и в GNUPlot); 3. Построить график изменения длины очереди и средней длины очереди на первом маршрутизаторе. 4. Оформить отчёт о выполненной работе.

2. Основная часть

2.1 Выполнение лабораторной работы

Описание моделируемой сети: – сеть состоит из N TCP-источников, N TCP-приёмников, двух маршрутизаторов $R1$ и $R2$ между источниками и приёмниками (N — не менее 20); – между TCP-источниками и первым маршрутизатором установлены дуплексные соединения с пропускной способностью 100 Мбит/с и задержкой 20 мс очередью типа DropTail; – между TCP-приёмниками и вторым маршрутизатором установлены дуплексные соединения с пропускной способностью 100 Мбит/с и задержкой 20 мс очередью типа DropTail; – между маршрутизаторами установлено симплексное соединение ($R1-R2$) с пропускной способностью 20 Мбит/с и задержкой 15 мс очередью типа RED, размером буфера 300 пакетов; в обратную сторону — симплексное соединение ($R2-R1$) с пропускной способностью 15 Мбит/с и задержкой 20 мс очередью типа DropTail; – данные передаются по протоколу FTP поверх TCP Reno; – параметры алгоритма RED: $q_{min} = 75$, $q_{max} = 150$, $q_w = 0,002$, $p_{max} = 0.1$; – максимальный размер TCP-окна 32; размер передаваемого пакета 500 байт; время моделирования — не менее 20 единиц модельного времени.

2.2 Начиная с написания лабораторной работы. С кодом можно ознакомиться в моем гитхабе. Прописываем код с подробными комментариями.

```
GNU nano 2.9.3 lab04.tcl

# Creating an object Simulator
set ns [new Simulator]

# Opening for record out.nam for nam-visualisation
set nf [open out.nam w]

# All the results will be recorded in variable nf
$ns namtrace-all $nf

# Opening for record a tracin-file out.tr for registration all of the results
set f [open out.tr w]

# All the registered results will be recorded in variable f
$ns trace-all $f

# Two nodes R1 and R2
set r1 [$ns node]
set r2 [$ns node]

# Parameters for TCP
Agent/TCP set window 32
Agent/TCP set packetSize_ 500

# Connecting between the nodes and queue size
$ns simplex-link $r1 $r2 20Mb 15ms RED
$ns simplex-link $r2 $r1 15Mb 20ms DropTail
$ns queue-limit $r1 $r2 300

# N=20
set N 20

# Adding agents and apps and FTP over TCP Reno
for {set i 0} {$i < $N} {incr i} {
    set n1($i) [$ns node]
    $ns duplex-link $n1($i) $r1 100Mb 20ms DropTail
    set n2($i) [$ns node]
    $ns duplex-link $n2($i) $r2 100Mb 20ms DropTail
}
```

I

2.3 Прописываем код с подробными комментариями.

```
    set tcp($i) [$ns create-connection TCP/Reno $n1($i) TCPSink $n2($i) $i]
    set ftp($i) [$tcp($i) attach-source FTP]
)

# Monitoring window size TCP
set windowVsTimeFirst [open windowVsTimeRenoFirst w]
puts $windowVsTimeFirst "0.Color: Black"

set windowVsTimeAll [open windowVsTimeRenoAll w]
puts $windowVsTimeAll "0.Color: Black"

set qmon [$ns monitor-queue $r1 $r2 [open qm.out w] 0.1];
[$ns link $r1 $r2] queue-sample-timeout;

# Monitoring queue
set redq [[$ns link $r1 $r2] queue]
$redq set thresh_ 75
$redq set maxthresh_ 150
```


2.4 Прописываем код с подробными комментариями.

```
# Monitoring queue
set redq [(sns link $r1 $r2) queue]
$redq set thresh_ 75
$redq set maxthresh_ 150
$redq set q_weight_ 0.002
$redq set linterm_ 10

set tchan_ [open all.q w]
$redq trace curq_
$redq trace ave_
$redq attach $tchan_

# Procedure finish
proc finish () {
    global tchan
    # Connecting code AWK
    set awkCode {
        {
            if ($1 == "Q" && NF>2) {
                print $2, $3 >> "temp.q";
                set end $2
            }
            else if ($1 == "a" && NF>2)
                print $2, $3 >> "temp.a";
        }
    }

    set f [open temp.q w]
    puts $f "0.Color: Black"
    close $f

    set f [open temp.a w]
    puts $f "0.Color: Black"
    close $f
}
```

2.5 Прописываем код с подробными комментариями.

```
GNU nano 2.9.3 lab04.tcl Изменён
# If files are already existing, we're deleting them and creating new
exec rm -f temp.q temp.a
exec touch temp.a temp.q

# Execution of code AWK
exec awk $awkCode all.q

# Starting Xgraph with graphics of the windows TCP and queues
exec xgraph -bb -tk -x t(s) -y CWIND(pkt) -t "TCPrenoCWNDOn1stLink" windowVsTimeRenoFirst -bg white -fg black &
exec xgraph -bb -tk -x t(s) -y CWIND(pkt) -t "TCPrenoCWNDOnAllSources" windowVsTimeRenoAll -bg white -fg black &
exec xgraph -bb -tk -x t(s) -y "Queue Length (pkt)" -t "ChangeOfSizeOfQueueOnLink(R1-R2)" temp.q -bg white -fg &
exec xgraph -bb -tk -x t(s) -y "Queue Avg Length (pkt)" -t "ChangeOfSizeOfAvgQueueOnLink(R1-R2)" temp.a -bg white &
exec nam out.nam &
exit 0
}

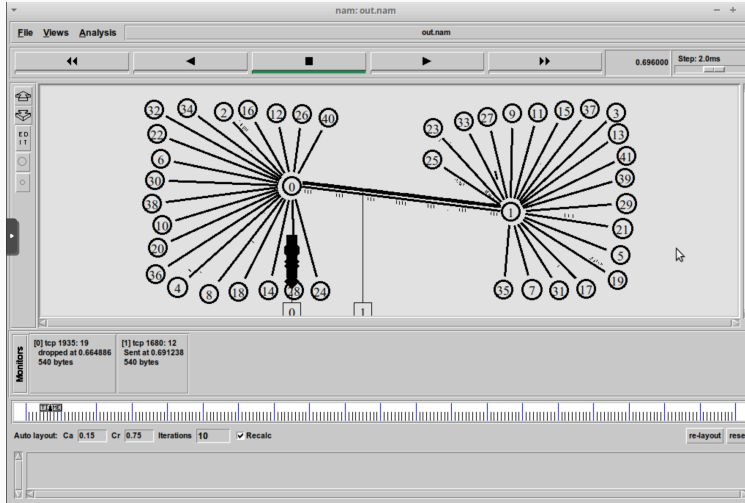
# Formation of file with TCP window size data
proc plotWindow {tcpSource file} {
    global ns
    set time 0.01
    set now [$ns now]
    set cwnd [$tcpSource set cwnd_]
    puts $file "$now $cwnd"
    $ns at [expr $now+$time] "plotWindow $tcpSource $file"
}

# Adding at-actions
for {set i 0} {$i < $N} {incr i} {
    $ns at 0.0 "$ftp($i) start"
    $ns at 0.0 "plotWindow $tcp($i) $windowVsTimeAll"
}
$ns at 0.0 "plotWindow $tcp(1) $windowVsTimeFirst"
$ns at 20.0 "finish"

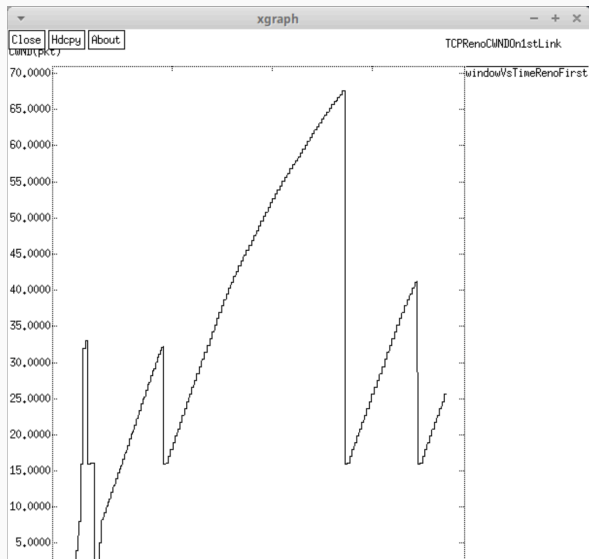
# Starting model
$ns run
```

⌘ Помощь ⌘ Записать ⌘ Поиск ⌘ Вырезать ⌘ Выводить ⌘ ТекПозиц ⌘ У Отмена ⌘ А Отметить
⌘ Выход ⌘ ЧитФайл ⌘ Замена ⌘ Отмен. вырезк ⌘ Словарь ⌘ К строке ⌘ Е Повтор ⌘ С Копировать

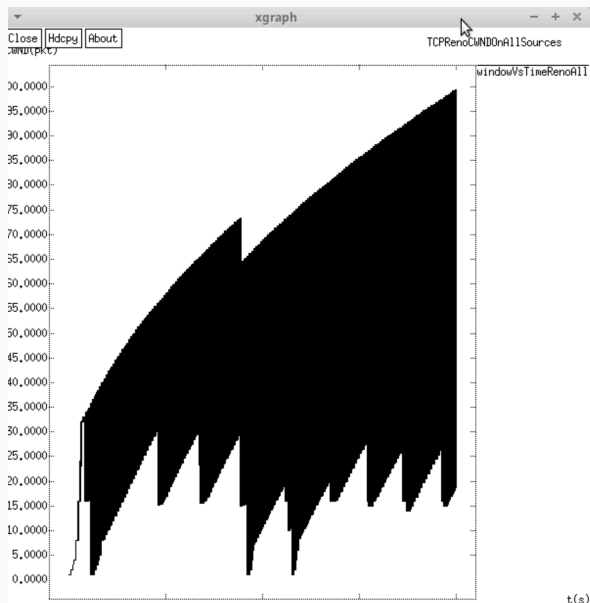
2.6 Получаем работающую модель, отмечены потерянные пакеты и сам трафик.



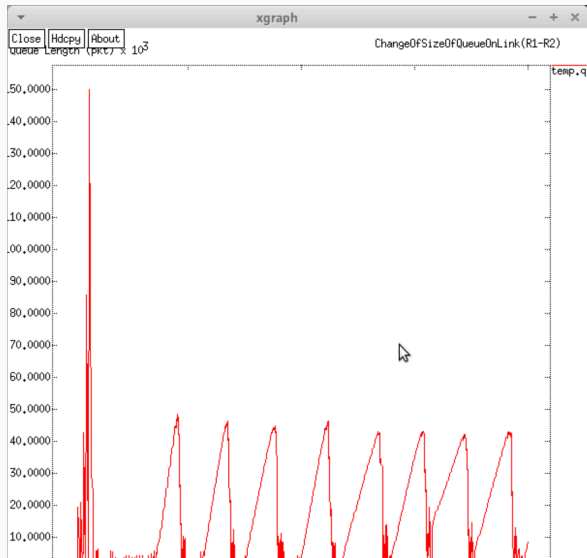
2.7 Код для отображения в Xgraph включен в код самой модели. Изменение размера окна TCP на линке 1-го источника при N=20.



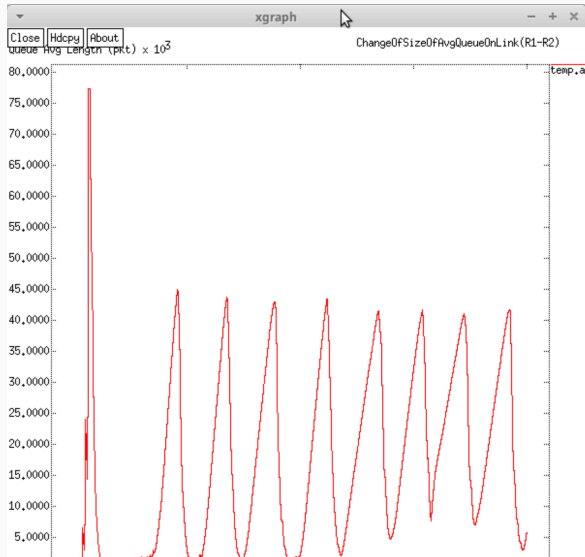
2.8 Изменение размера окна TCP на всех источниках при N=20.



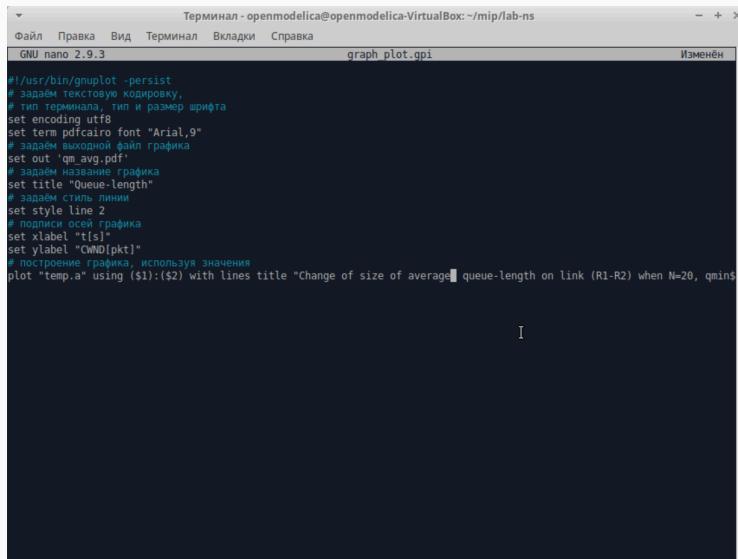
2.9 Изменение размера длины очереди на линке (R1–R2) при $N=20$, $q_{\min} = 75$, $q_{\max} = 150$.



2.10 Изменение размера средней длины очереди на линке (R1–R2) при $N=20$, $q_{\min} = 75$, $q_{\max} = 150$.

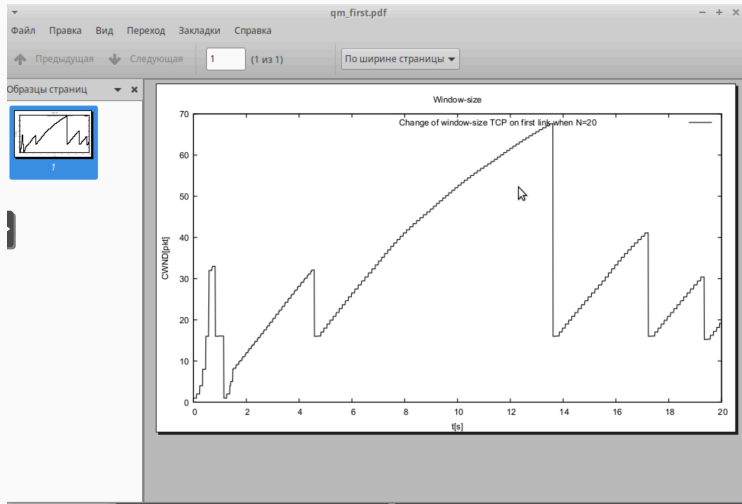


2.11 Теперь сделаем такие же графики в GNUPlot. Для этого меняем наш скрипт graph_plot.gpi из прошлой лабораторной работы.

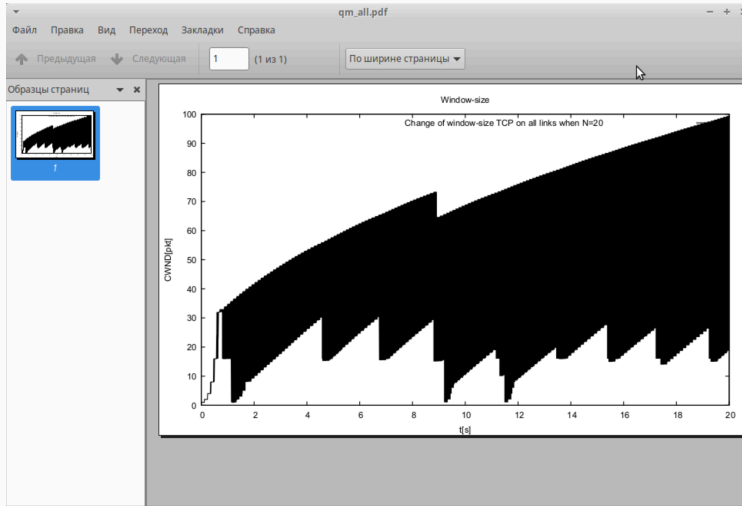


```
Терминал - openmodelica@openmodelica-VirtualBox: ~/mip/lab-ns
Файл  Правка  Вид  Терминал  Вкладки  Справка
GNU nano 2.9.3 graph_plot.gpi Изменён
#!/usr/bin/gnuplot -persist
# задаём текстовую кодировку,
# тип терминала, тип и размер шрифта
set encoding utf8
set term pdfcairo font "Arial,9"
# задаём выходной файл графика
set out 'qm_avg.pdf'
# задаём название графика
set title "Queue-length"
# задаём стиль линии
set style line 2
# подписи осей графика
set xlabel "t[s]"
set ylabel "CWND[pkt]"
# построение графика, используя значения
plot "temp.a" using ($1):($2) with lines title "Change of size of average queue-length on link (R1-R2) when N=20, qmins"
```

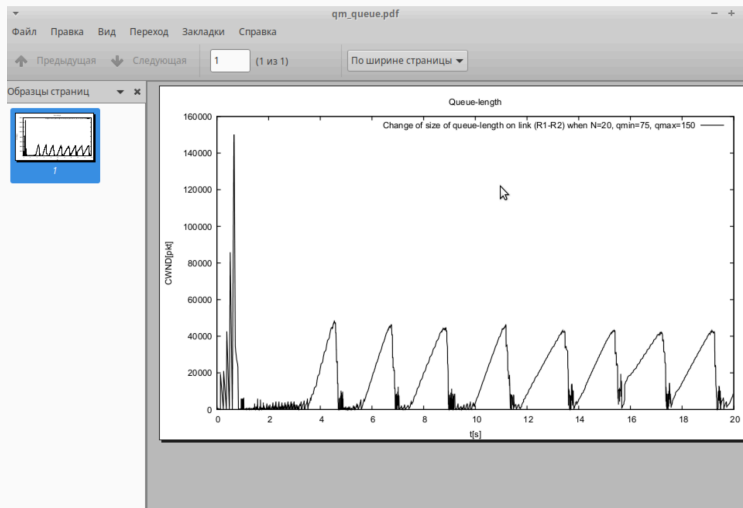

2.12 Изменение размера окна TCP на линке 1-го источника при $N=20$.



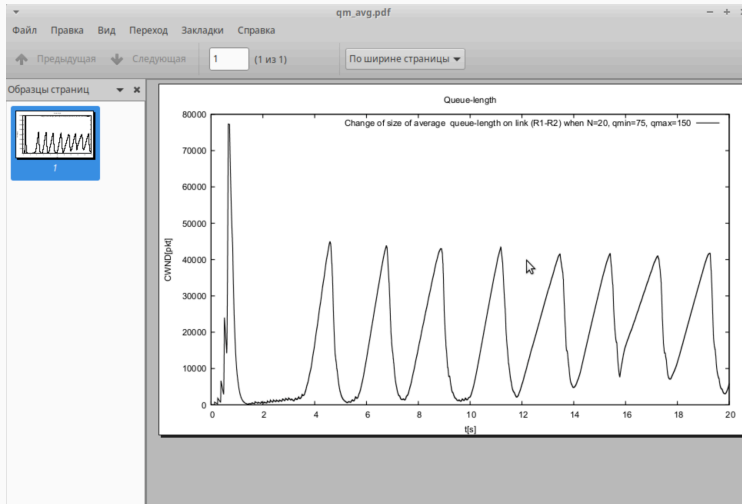
2.13 Изменение размера окна TCP на всех источниках при N=20.



2.14 Изменение размера длины очереди на линке (R1-R2) при $N=20$, $q_{\min} = 75$, $q_{\max} = 150$.



2.15 Изменение размера средней длины очереди на линке (R1–R2) при $N=20$, $q_{\min} = 75$, $q_{\max} = 150$.



3. Результаты

3. Результаты

Мы смогли с помощью ранее полученных шаблонов и знаний реализовать сложную модель, наглядно продемонстрировать результаты ее работы и закончили прохождение данной темы.